

Documento de Especificación de Requerimientos de Software
(SRS)

Plataforma Automatizada de Auditoría y Mapeo de Redes Corporativas

Primera Iteración – Producto Mínimo Viable (MVP)

Documentación Oficial de Portafolio Técnico

16 de julio de 2026

Código de Documento	SRS-PAMRC-001
Versión	1.0
Estado	Borrador para desarrollo – Iteración 1
Clasificación	Documentación técnica de portafolio profesional
Stack Tecnológico Principal	Python, Node.js, Supabase (PostgreSQL), Astro
Equipo de Desarrollo	1 desarrolladora (Ing. en Sistemas, en formación activa)
Capacidad de Desarrollo	2 horas diarias (~14 hrs/semana)
Entorno Local de Desarrollo	AMD Ryzen 5 8645HS / GPU dedicada RTX 3050
Gestión del Proyecto	Notion (seguimiento visual) / Git (control de versiones)

Índice

1. Resumen Ejecutivo	3
2. Objetivos del Proyecto	3
2.1. Objetivo General	3
2.2. Objetivos Específicos	3
3. Alcance y Límites (Scope)	4
3.1. Incluido en esta Iteración (Dentro del Alcance)	4
3.2. Fuera del Alcance (Out of Scope)	4
4. Requerimientos Funcionales	4
4.1. Módulo RF-PY: Motor de Escaneo (Python)	5
4.2. Módulo RF-ND: API de Transporte (Node.js)	5
4.3. Módulo RF-DB: Persistencia de Datos (Supabase / PostgreSQL)	6
4.4. Módulo RF-AS: Dashboard Web (Astro)	6
5. Requerimientos No Funcionales	6
5.1. Rendimiento	6
5.2. Seguridad	7
5.3. Mantenibilidad	7
6. Arquitectura de Datos a Alto Nivel	7
6.1. Capa 1: Recolección (Python)	7
6.2. Capa 2: Transporte (Node.js)	8
6.3. Capa 3: Persistencia (Supabase / PostgreSQL)	8
6.4. Capa 4: Presentación (Astro)	8
7. Plan de Desarrollo y Cronograma Detallado	8
7.1. Fase 1: Motor de Escaneo en Python	8
7.2. Fase 2: Diseño e Implementación de Base de Datos (Supabase)	9
7.3. Fase 3: API de Transporte en Node.js	10
7.4. Fase 4: Dashboard Web en Astro	10
7.5. Resumen Consolidado del Cronograma	11

1 Resumen Ejecutivo

En la mayoría de las infraestructuras corporativas de tamaño pequeño y mediano, el crecimiento orgánico de la red genera un problema recurrente: la pérdida progresiva de visibilidad sobre la topología real de segmentación. Switches configurados de forma ad-hoc, VLANs mal documentadas y dispositivos que se incorporan a la red sin registro formal derivan en un mapa de red desactualizado respecto a la realidad operativa. Esta brecha entre la documentación teórica y el estado real de la red constituye una superficie de riesgo significativa: dificulta la respuesta a incidentes, complica las auditorías de cumplimiento (ISO 27001, PCI-DSS) y oculta rutas de movimiento lateral que un atacante podría explotar.

La **Plataforma Automatizada de Auditoría y Mapeo de Redes Corporativas** (en adelante, PAMRC) nace como respuesta directa a esta problemática. Se trata de un sistema modular cuyo objetivo es escanear de forma activa los segmentos de red accesibles, identificar hosts, servicios expuestos y relaciones de conectividad, y traducir esa información cruda en un mapa visual e interpretable, disponible en un dashboard web centralizado.

Esta primera iteración corresponde a un **Producto Mínimo Viable (MVP)**, concebido con un doble propósito: (1) validar técnicamente el flujo completo de datos –desde la recolección hasta la visualización– y (2) servir como pieza central de portafolio técnico, demostrando dominio práctico de un stack políglota (Python, Node.js, Supabase/PostgreSQL y Astro) integrado bajo una arquitectura coherente y documentada.

El desarrollo se ejecuta bajo restricciones reales y declaradas: una única desarrolladora en formación activa, con una disponibilidad de dos horas diarias, trabajando sobre un equipo de cómputo local de gama media-alta (AMD Ryzen 5 8645HS, GPU RTX 3050), utilizando Notion para la gestión visual del backlog y Git como mecanismo estricto de control de versiones. Este documento traduce dichas restricciones en un plan de desarrollo realista, secuencial y verificable.

2 Objetivos del Proyecto

2.1 Objetivo General

Diseñar, desarrollar e implementar una plataforma modular capaz de escanear redes corporativas de forma automatizada, persistir los hallazgos en una base de datos relacional centralizada y presentar la topología resultante mediante un dashboard web funcional, estableciendo así una base técnica escalable para futuras iteraciones de auditoría continua.

2.2 Objetivos Específicos

- OE-1** Construir un motor de escaneo de red en Python capaz de realizar descubrimiento de hosts activos, identificación de puertos abiertos y detección básica de servicios dentro de un segmento de red definido por el usuario.
- OE-2** Desarrollar una API de transporte en Node.js que actúe como intermediaria segura entre el motor de escaneo y la base de datos, exponiendo endpoints RESTful para la ingesta, consulta y consumo de los datos de auditoría.
- OE-3** Diseñar e implementar un esquema de base de datos relacional en Supabase (PostgreSQL) que modele de forma normalizada los activos de red, sus relaciones de conectividad y el historial de escaneos ejecutados.
- OE-4** Implementar un dashboard web en Astro que consuma la API de transporte y permita visualizar, de forma clara y navegable, el mapa de red generado y los detalles de cada activo identificado.

3 Alcance y Límites (Scope)

3.1 Incluido en esta Iteración (Dentro del Alcance)

- Escaneo de un único segmento de red (rango CIDR) por ejecución, definido manualmente por el usuario.
- Descubrimiento de hosts activos mediante técnicas estándar (ICMP / ARP para red local).
- Escaneo de puertos TCP comunes y detección básica de servicios (banner grabbing simple).
- Persistencia de resultados en Supabase con historial mínimo de ejecuciones (timestamp por escaneo).
- API REST con operaciones CRUD básicas sobre activos y escaneos.
- Autenticación simple de la API mediante clave compartida (API Key) gestionada por variables de entorno.
- Dashboard web con listado de activos, vista de detalle por host y una visualización topológica básica (grafo simple).
- Ejecución manual del escaneo (disparado por el usuario desde el dashboard o vía CLI).
- Documentación técnica del código y del flujo de arquitectura como parte del entregable de portafolio.

3.2 Fuera del Alcance (Out of Scope)

- Escaneo de múltiples segmentos de red de forma simultánea o distribuida.
- Detección de vulnerabilidades (CVE matching) o integración con bases de datos de exploits.
- Escaneo programado / automatizado por cron o triggers recurrentes (queda para iteración 2).
- Autenticación multiusuario, roles y permisos granulares (RBAC).
- Notificaciones en tiempo real (correo, Slack, webhooks) ante cambios en la topología.
- Soporte para protocolos IPv6, SNMP o descubrimiento de dispositivos vía protocolos propietarios de fabricantes (Cisco Discovery Protocol, etc.).
- Despliegue en infraestructura cloud productiva (la iteración 1 se limita a entorno local / desarrollo).
- Generación de reportes en PDF o exportación de cumplimiento normativo.

4 Requerimientos Funcionales

Los requerimientos funcionales se agrupan por módulo tecnológico para mantener trazabilidad directa entre la especificación y la fase de desarrollo correspondiente.

4.1 Módulo RF-PY: Motor de Escaneo (Python)

ID	Descripción	Prioridad
RF-PY-01	El sistema debe permitir al usuario definir un rango de red en notación CIDR (ej. 192.168.1.0/24) como parámetro de entrada del escaneo.	Alta
RF-PY-02	El motor debe identificar hosts activos dentro del rango especificado mediante técnicas de descubrimiento (ping sweep / ARP request).	Alta
RF-PY-03	El motor debe escanear un conjunto configurable de puertos TCP (por defecto, los 1000 más comunes) por cada host activo detectado.	Alta
RF-PY-04	El sistema debe intentar identificar el servicio asociado a cada puerto abierto mediante lectura de banner (banner grabbing).	Media
RF-PY-05	El motor debe estructurar los resultados en un formato serializable (JSON) previo a su envío hacia la API de transporte.	Alta
RF-PY-06	El sistema debe registrar en consola (logging) el progreso y los errores del escaneo en ejecución, para fines de depuración.	Media
RF-PY-07	El motor debe enviar los resultados consolidados a la API de Node.js mediante una petición HTTP autenticada.	Alta

4.2 Módulo RF-ND: API de Transporte (Node.js)

ID	Descripción	Prioridad
RF-ND-01	La API debe exponer un endpoint POST <code>/scans</code> para recibir e ingestar los resultados enviados por el motor Python.	Alta
RF-ND-02	La API debe validar la autenticidad de la petición mediante una API Key transmitida en las cabeceras HTTP.	Alta
RF-ND-03	La API debe exponer un endpoint GET <code>/assets</code> que retorne el listado de activos de red registrados.	Alta
RF-ND-04	La API debe exponer un endpoint GET <code>/assets/:id</code> que retorne el detalle completo de un activo, incluyendo puertos y servicios asociados.	Alta
RF-ND-05	La API debe exponer un endpoint GET <code>/scans/latest</code> que retorne el resumen del escaneo más reciente ejecutado.	Media
RF-ND-06	La API debe validar y sanear (sanitize) todo dato de entrada antes de persistirlo en la base de datos, previniendo inyección SQL.	Alta
RF-ND-07	La API debe centralizar la comunicación con Supabase mediante el cliente oficial o consultas parametrizadas, evitando acceso directo desde otros módulos.	Alta

4.3 Módulo RF-DB: Persistencia de Datos (Supabase / PostgreSQL)

ID	Descripción	Prioridad
RF-DB-01	El esquema debe contar con una tabla assets que almacene IP, hostname (si se resuelve), primer y último visto.	Alta
RF-DB-02	El esquema debe contar con una tabla ports relacionada a assets , almacenando número de puerto, protocolo y estado.	Alta
RF-DB-03	El esquema debe contar con una tabla services relacionada a ports , almacenando el nombre del servicio detectado y su banner crudo.	Alta
RF-DB-04	El esquema debe contar con una tabla scans que registre cada ejecución de escaneo con su timestamp y rango CIDR utilizado.	Alta
RF-DB-05	Las relaciones entre tablas deben estar normalizadas (mínimo 3FN) y protegidas mediante llaves foráneas con integridad referencial.	Alta
RF-DB-06	El esquema debe permitir consultar el historial de cambios de un activo a través de múltiples escaneos (comparación temporal básica).	Media
RF-DB-07	El acceso a la base de datos debe estar restringido mediante políticas de Row Level Security (RLS) de Supabase, incluso en esta iteración inicial.	Alta

4.4 Módulo RF-AS: Dashboard Web (Astro)

ID	Descripción	Prioridad
RF-AS-01	El dashboard debe presentar una vista principal con el listado de activos de red obtenidos del último escaneo.	Alta
RF-AS-02	El dashboard debe permitir acceder a una vista de detalle por activo, mostrando puertos abiertos y servicios detectados.	Alta
RF-AS-03	El dashboard debe incluir una visualización topológica básica (grafo de nodos y conexiones) representando la red escaneada.	Media
RF-AS-04	El dashboard debe consumir exclusivamente la API de Node.js, sin acceso directo a Supabase desde el cliente.	Alta
RF-AS-05	El dashboard debe mostrar el timestamp del último escaneo ejecutado y un indicador de estado (actualizado / desactualizado).	Media
RF-AS-06	El dashboard debe ser responsivo, garantizando legibilidad mínima en resoluciones de escritorio y tablet.	Media

5 Requerimientos No Funcionales

5.1 Rendimiento

- El motor de escaneo debe completar un análisis de una red /24 (254 hosts) en un tiempo razonable de ejecución local, aprovechando paralelización controlada (threading o `asyncio`) sin saturar el hardware disponible (AMD Ryzen 5 8645HS).
- El consumo de CPU durante el escaneo no debe impedir el uso simultáneo del equipo para tareas de desarrollo (edición de código, compilación), dado que el entorno de pruebas y desarrollo es el mismo equipo local.
- La GPU dedicada (RTX 3050) no es requerida por el MVP; su disponibilidad se reserva para posibles módulos futuros de análisis o clasificación (fuera del alcance de esta iteración).

- Las respuestas de la API de Node.js hacia el dashboard deben resolverse en un tiempo perceptible como inmediato para volúmenes de datos propios del MVP (cientos de registros, no miles).

5.2 Seguridad

- Todas las credenciales sensibles (claves de Supabase, API Keys internas, cadenas de conexión) deben gestionarse exclusivamente mediante archivos `.env`, excluidos del control de versiones mediante `.gitignore`.
- Ningún secreto debe quedar hardcodeado en el código fuente en ningún módulo (Python, Node.js o Astro).
- La comunicación entre el motor Python y la API Node.js debe validarse mediante autenticación por clave compartida como mínimo viable de seguridad para esta iteración.
- El dashboard Astro no debe exponer directamente las credenciales de Supabase al cliente; toda consulta debe pasar por la API de Node.js.
- Se debe mantener un archivo `.env.example` documentado como referencia de variables requeridas, sin valores reales.

5.3 Mantenibilidad

- El código debe organizarse en módulos claramente separados por responsabilidad (escaneo, transporte, persistencia, presentación), favoreciendo el principio de separación de capas.
- Cada módulo debe contar con un archivo `README.md` propio que documente su propósito, instalación y ejecución local.
- El control de versiones debe seguir una convención estricta de commits (se recomienda Conventional Commits) y ramas por funcionalidad (`feature/`, `fix/`), integradas mediante Pull Requests hacia `main`.
- El seguimiento de tareas en Notion debe reflejar el estado real del tablero de Git (backlog, en progreso, en revisión, completado) para mantener trazabilidad entre gestión y ejecución técnica.
- El esquema de base de datos debe versionarse mediante migraciones (SQL o herramientas de migración de Supabase), evitando cambios manuales no documentados.

6 Arquitectura de Datos a Alto Nivel

La arquitectura del sistema se organiza en cuatro capas secuenciales, diseñadas para mantener un flujo de datos unidireccional y desacoplado, facilitando tanto el desarrollo incremental como la depuración de fallos por capa.

6.1 Capa 1: Recolección (Python)

El motor de escaneo, escrito en Python, actúa como el punto de origen de los datos. Recibe como entrada un rango de red en notación CIDR, ejecuta el descubrimiento de hosts activos y el escaneo de puertos/servicios, y consolida los hallazgos en una estructura de datos serializable (JSON). Esta capa no tiene conocimiento directo de la base de datos; su única responsabilidad es recolectar y entregar datos crudos.

6.2 Capa 2: Transporte (Node.js)

La API construida en Node.js recibe los datos generados por la capa de recolección a través de un endpoint autenticado. Esta capa actúa como intermediaria y guardiana: valida, sana y transforma los datos antes de persistirlos, y además expone los endpoints de consulta que consumirá el dashboard. Ningún otro módulo del sistema se comunica directamente con la base de datos; toda escritura y lectura pasa obligatoriamente por esta capa.

6.3 Capa 3: Persistencia (Supabase / PostgreSQL)

Los datos validados se almacenan en un esquema relacional normalizado sobre Supabase (PostgreSQL). Esta capa es responsable de mantener la integridad referencial entre activos, puertos, servicios y escaneos, además de aplicar políticas de seguridad a nivel de fila (RLS). Su diseño contempla la posibilidad de consultas históricas para futuras iteraciones de comparación temporal.

6.4 Capa 4: Presentación (Astro)

El dashboard web, construido en Astro, consume exclusivamente la API de Node.js para renderizar la información en interfaces comprensibles: listados de activos, vistas de detalle y una representación topológica básica. Esta capa no accede a la base de datos ni al motor de escaneo de forma directa, preservando el aislamiento entre presentación y lógica de negocio.

Resumen del flujo:

Python (Escaneo) → Node.js (API/Transporte) → Supabase (Persistencia) → Node.js (Consulta) → Astro (Presentación)

7 Plan de Desarrollo y Cronograma Detallado

El siguiente plan traduce el alcance definido en un cronograma secuencial de cuatro fases, alineadas directamente con las capas de arquitectura descritas. Las estimaciones de tiempo consideran una disponibilidad real de **2 horas diarias** por parte de la desarrolladora, expresadas en **semanas de esfuerzo** (asumiendo aproximadamente 14 horas efectivas de trabajo por semana). Cada fase incluye una guía de actividades paso a paso, utilizable directamente como plan de estudio e implementación, y culmina en un hito de entrega verificable.

7.1 Fase 1: Motor de Escaneo en Python

Duración estimada: 3 semanas de esfuerzo (~42 horas).

Actividades paso a paso:

1. Configurar el entorno virtual de Python (`venv`) e inicializar el repositorio Git del módulo con estructura de carpetas base.
2. Investigar e implementar descubrimiento de hosts activos en una red local (ping sweep con `socket` o librería `scapy`).
3. Implementar el escaneo de puertos TCP sobre los hosts detectados, iniciando con un enfoque secuencial simple.
4. Optimizar el escaneo mediante concurrencia (`threading` o `asyncio`) para reducir el tiempo total de ejecución.

5. Implementar banner grabbing básico sobre los puertos abiertos identificados para inferir el servicio.
6. Estructurar los resultados en un diccionario/JSON consistente, definiendo el contrato de datos que se enviará a la API.
7. Implementar logging estructurado (módulo `logging`) para trazabilidad de la ejecución.
8. Escribir pruebas unitarias básicas para las funciones de parsing y estructuración de resultados.
9. Documentar el módulo en un `README.md` (instalación, uso por CLI, parámetros aceptados).
10. Realizar commit final de la fase siguiendo la convención de Conventional Commits y fusionar a `main` mediante Pull Request.

Hito de Entrega: El motor de escaneo ejecuta de forma autónoma vía línea de comandos, recibe un rango CIDR como parámetro, y genera un archivo JSON local con la lista de hosts activos, puertos abiertos y servicios detectados, sin necesidad aún de conexión a la API o base de datos.

7.2 Fase 2: Diseño e Implementación de Base de Datos (Supabase)

Duración estimada: 2 semanas de esfuerzo (~28 horas).

Actividades paso a paso:

1. Crear el proyecto en Supabase y familiarizarse con el panel de administración y el editor SQL.
2. Diseñar el modelo entidad-relación (ER) para las tablas `assets`, `ports`, `services` y `scans`.
3. Implementar el esquema mediante migraciones SQL, definiendo llaves primarias, foráneas y restricciones de integridad.
4. Normalizar el esquema hasta Tercera Forma Normal (3FN), validando la ausencia de redundancias.
5. Configurar políticas de Row Level Security (RLS) básicas para restringir el acceso directo a las tablas.
6. Insertar datos de prueba (seed data) manualmente para validar las relaciones antes de conectar la API.
7. Escribir y probar consultas SQL clave (últimos activos, historial de un host, resumen de un escaneo).
8. Documentar el esquema (diagrama ER y diccionario de datos) dentro del repositorio.
9. Versionar las migraciones en Git y fusionar la fase mediante Pull Request.

Hito de Entrega: El esquema relacional está desplegado en Supabase, normalizado, con políticas de seguridad activas y validado mediante datos de prueba y consultas SQL documentadas, listo para recibir datos reales desde la API.

7.3 Fase 3: API de Transporte en Node.js

Duración estimada: 3 semanas de esfuerzo (~42 horas).

Actividades paso a paso:

1. Inicializar el proyecto Node.js (`npm init`), instalar dependencias base (framework HTTP, cliente de Supabase, manejo de variables de entorno).
2. Configurar el archivo `.env` y su correspondiente `.env.example`, integrando las credenciales de Supabase de forma segura.
3. Implementar el middleware de autenticación mediante API Key para proteger los endpoints sensibles.
4. Desarrollar el endpoint `POST /scans` para la ingesta de resultados provenientes del motor Python, incluyendo validación y saneamiento de datos.
5. Desarrollar los endpoints de consulta: `GET /assets`, `GET /assets/:id` y `GET /scans/latest`.
6. Integrar el motor Python con la API mediante una petición HTTP real (por ejemplo, con `requests`), cerrando el flujo de las Capas 1 y 2.
7. Implementar manejo centralizado de errores y respuestas HTTP consistentes (códigos de estado adecuados).
8. Escribir pruebas de integración básicas para los endpoints principales.
9. Documentar la API (rutas, parámetros, ejemplos de request/response) en el `README.md` del módulo.
10. Fusionar la fase a `main` mediante Pull Request, validando que el flujo completo Python → Node.js → Supabase funcione de extremo a extremo.

Hito de Entrega: Al ejecutar el motor de escaneo, los resultados se transmiten automáticamente a la API de Node.js, se validan y se persisten correctamente en Supabase; los endpoints de consulta retornan datos reales verificables mediante herramientas como Postman o Thunder Client.

7.4 Fase 4: Dashboard Web en Astro

Duración estimada: 3 semanas de esfuerzo (~42 horas).

Actividades paso a paso:

1. Inicializar el proyecto Astro y definir la estructura base de páginas y componentes.
2. Implementar el cliente de consumo de la API de Node.js (fetch a los endpoints `/assets` y `/scans/latest`).
3. Construir la vista principal con el listado de activos de red, incluyendo estado y timestamp del último escaneo.
4. Construir la vista de detalle por activo, mostrando puertos abiertos y servicios detectados.
5. Integrar una librería de visualización de grafos (evaluando opciones ligeras compatibles con Astro) para representar la topología básica.
6. Aplicar estilos responsivos, garantizando legibilidad en resoluciones de escritorio y tablet.

7. Implementar manejo de estados de carga y error en las vistas (mientras se consulta la API).
8. Realizar pruebas manuales de extremo a extremo: ejecutar escaneo real, verificar persistencia y validar su correcta visualización en el dashboard.
9. Documentar el módulo (instalación, variables de entorno requeridas, comandos de build/dev) en su `README.md`.
10. Consolidar el `README.md` general del proyecto (raíz del repositorio) integrando la documentación de las cuatro fases y fusionar mediante Pull Request final.

Hito de Entrega: El dashboard Astro se despliega localmente, consume la API de Node.js sin acceso directo a Supabase, y permite visualizar –de forma navegable y comprensible– el mapa de red generado por un escaneo real, cerrando el ciclo completo de las cuatro capas de arquitectura y marcando la conclusión funcional del MVP.

7.5 Resumen Consolidado del Cronograma

Fase	Módulo	Duración	Acumulado
1	Motor de Escaneo (Python)	3 semanas	3 semanas
2	Base de Datos (Supabase)	2 semanas	5 semanas
3	API de Transporte (Node.js)	3 semanas	8 semanas
4	Dashboard Web (Astro)	3 semanas	11 semanas
Duración total estimada del MVP		11 semanas (~154 horas)	

Nota metodológica: esta estimación asume una ejecución secuencial estricta de las fases, sin bloqueos externos ni retrabajo mayor. Se recomienda mantener en Notion un buffer de contingencia del 15-20 % sobre esta duración total, dado que corresponde a un desarrollo individual con curva de aprendizaje activa en al menos dos de las cuatro tecnologías del stack.